

PROJECT REPORT

Project Title: Opsis AI - Intelligent Healthcare Assistant

Institution: Haldia Institute of Technology

Department: Information Technology (B.Tech)

1. Abstract

In recent years, the integration of Artificial Intelligence in healthcare has transformed how medical data is analyzed. **Opsis AI** is a full-stack web application designed to act as a personal, intelligent healthcare assistant. The primary goal of this project is to bridge the gap between complex medical terminologies and common users. By leveraging Generative AI, Opsis AI can analyze complex medical PDF reports (like blood test panels) and translate them into simple, easy-to-understand health scores and actionable insights. Additionally, it features an AI-driven Symptom Checker that acts as a primary triage system, guiding users on the urgency of their health conditions.

2. Introduction

2.1 Problem Statement

When patients receive lab test results, the reports are often filled with medical jargon, complex metrics, and numbers that are difficult for a non-medical person to understand. Similarly, when people experience symptoms, searching the internet often leads to unnecessary panic due to generic or extreme search results.

2.2 Proposed Solution

Opsis AI solves this problem by providing a user-friendly platform where users can safely input their symptoms or upload their medical reports. The system securely processes the text, communicates with an AI model, and returns a structured, summarized, and simple explanation of the user's health status, including a "Health Score", flagged biomarkers, and basic self-care advice.

3. Technology Stack

The application is built using the modern **MERN** stack combined with Generative AI tools and robust deployment platforms.

- **Frontend:** React.js, Vite, Tailwind CSS (for responsive UI), Framer Motion (for animations).
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB (for storing user sessions and report history).

- **AI & Libraries:** * pdfjs-dist: For extracting raw text from uploaded PDF files.
 - html2canvas & jspdf: For generating downloadable PDF summaries.
 - jsonwebtoken: For secure user session management.
- **Deployment & Hosting:**
 - **Frontend:** Netlify (Custom Domain: opsisai.netlify.app)
 - **Backend:** Render cloud platform.

4. System Architecture

The project follows a standard Client-Server architecture:

1. **Client Interface:** The user interacts with the React frontend. They can either fill out a symptom form or upload a PDF report.
2. **Text Extraction:** If a PDF is uploaded, the frontend uses pdfjs-dist to extract the text locally to save backend bandwidth and improve speed.
3. **API Processing:** The data is sent to the Node.js backend via secure API endpoints (/api/symptoms/analyze or /api/reports/analyze).
4. **AI Generation:** The backend formulates a strict prompt and sends it to the AI model to generate a structured JSON response containing the medical analysis.
5. **Database Storage:** The parsed result is saved in MongoDB for future retrieval.
6. **Data Presentation:** The frontend receives the data and displays it using interactive dashboards, circular progress rings (Health Score), and color-coded urgency badges.

5. Key Features

- **AI Symptom Checker (Triage):** Users enter their symptoms, and the AI categorizes the urgency level (Emergency, Urgent, Moderate, Low), suggests possible conditions, and provides immediate self-care actions.
- **Medical Report Analyzer:** Users upload lab reports. The app reads the PDF, flags abnormal biomarkers (e.g., Low Vitamin D, High Cholesterol), and suggests dietary or medical regimens.
- **Interactive Dashboard:** A clean, minimalistic user interface that displays a calculated "Health Score" out of 100.
- **Export functionality:** Users can click a button to instantly download their simplified AI report as a beautifully formatted PDF.

6. Challenges Faced and Solutions

Building and deploying a full-stack application presented several real-world engineering challenges, which were successfully resolved:

- **Challenge 1: Server Cold Starts (Spin-down)**
 - *Issue:* The backend hosted on Render's free tier would go to "sleep" after 15 minutes of inactivity, causing a 50-second delay for the next user.

- *Solution:* Implemented a `/api/health` endpoint and configured **UptimeRobot** to ping the server every 5 minutes, ensuring the server remains awake 24/7 for a seamless user experience.
- **Challenge 2: Cross-Origin Resource Sharing (CORS) Blocks**
 - *Issue:* The backend refused connections from the newly deployed Netlify frontend domain.
 - *Solution:* Configured the Express CORS middleware to dynamically trust requests coming specifically from the custom production domain.
- **Challenge 3: Single Page Application (SPA) Routing Errors**
 - *Issue:* Refreshing the page on the deployed Netlify site resulted in a "404 Page Not Found" error because the server was looking for physical HTML files for different routes.
 - *Solution:* Created a `_redirects` file in the public directory to route all traffic back to `index.html`, allowing React Router to handle the navigation smoothly.
- **Challenge 4: PDF Font Extraction Errors**
 - *Issue:* The `pdfjs-dist` library crashed when trying to read PDFs that did not have standard fonts embedded inside them.
 - *Solution:* Dynamically injected a `standardFontDataUrl` during the PDF parsing process, allowing the app to download necessary fonts on the fly and parse any medical report without crashing.

7. Future Scope

While the current version of Opsis AI is fully functional, future updates can include:

- **User Accounts:** Implementing full authentication (Login/Signup) to allow users to maintain a timeline of their medical history across multiple years.
- **Multilingual Support:** Translating the AI output into regional languages (e.g., Bengali, Hindi) to make it accessible to rural populations.
- **Doctor Integration:** A feature to directly share the AI-summarized report with registered medical practitioners for remote consultations.

8. Conclusion

The development of Opsis AI successfully demonstrates how modern web development technologies and Artificial Intelligence can be combined to solve real-world problems. The project provides a scalable, secure, and user-friendly platform that empowers individuals to take control of their health data. By overcoming various deployment and architectural challenges, this project serves as a strong foundation for a production-ready healthcare application.